

SQL Pivot and Cross Tab

By Don Schlichting

This article will explore various SQL methods for producing pivot style reports.

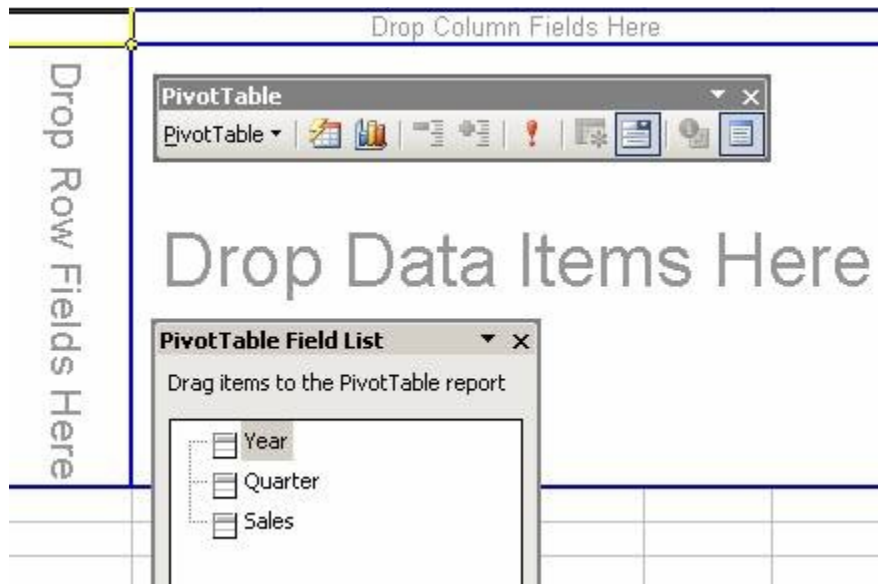
Introduction

A pivot is a data mining option typically used to transform large amounts data into a condensed list. Pivot style reports display data in a grid along a vertical and horizontal axis. Financial and managerial reports are the examples usually cited. The pivot style report pictured below displays sales data; with the Year of the sales running along one axis and the Quarter the sales was made running along the other. The sales amount is the sum of all dollars for the particular year and quarter.

If a regular select were used with a group by, the result would resemble this next image. Notice the amount of rows will increase. The pivot displayed three rows, one row for each year of data. Twelve rows would be needed in the select example to display the same information. This makes the pivot style more readable, and its format usually expected in business reporting.

YearSold	Quarter	Amount
2003	Q1	1.00
2003	Q2	2.00
2003	Q3	3.00
2003	Q4	4.00
2004	Q1	5.00
2004	Q2	6.00
2004	Q3	7.00
2004	Q4	8.00

For Microsoft Office users, Pivot reports and tables are standard options. In Microsoft Excel, there is a drag and drop wizard that walks a user through all the steps needed to create a pivot table. The final result is the grid displayed in the first image. Unlike a simple pivot report, Excel Pivot Tables can have their Column and Rows dynamically changed by dragging the desired column to the correct axis.



In Microsoft Access, there is a wizard for creating pivot queries, called "Cross tabs." The wizard asks which columns should be on the axis, then auto-generates the code needed. Access creates pivots by using the special key word TRANSFORM:

```
TRANSFORM Sum(SALES.AMOUNT) AS SumOfAMOUNT
SELECT SALES.YEAR, Sum(SALES.AMOUNT) AS [Total Of AMOUNT]
FROM SALES
GROUP BY SALES.YEAR
PIVOT SALES.QUARTER;
```

Unfortunately, there is no wizard for producing T-SQL Pivots. In SQL 2000, pivot style reports, called "Cross-Tab Reports" can be created using the CASE keyword. In SQL 2005, CASE can be used, or there is a new keyword, PIVOT.

SQL Pivot and Cross Tab - Page 2

By Don Schlichting

CASE

The keyword CASE can be used to build pivots in SQL Server. SQL Server Books On Line will call reports generated by a CASE statement as Cross Tab Reports. Both terms introduced so far, Cross Tab and Pivot, imply the same type of end result, a grid with an X Y axis and populated with summarized data. The main purpose of CASE in T-SQL is to allow an alternate value to be displayed during a Select statement. This example creates a small table holding state abbreviations. CASE will be used to display the state name.

```
CREATE TABLE #States
(
  StateCode char(2)
```

```

)
INSERT INTO #States VALUES ('WI')
INSERT INTO #States VALUES ('OH')
INSERT INTO #States VALUES ('CA')
SELECT StateCode,
CASE StateCode
WHEN 'WI' THEN 'Wisconsin'
WHEN 'OH' THEN 'OHIO'
WHEN 'CA' THEN 'California'
END AS State_Name
FROM #States
DROP TABLE #States

```

StateCode	State_Name
WI	Wisconsin
OH	OHIO
CA	California

The CASE syntax is uncomplicated. WHEN a condition is true, CASE will display the alternative provided after the THEN.

CASE and Cross Tab Reports

When CASE is used to create a pivot style report, BOL calls it a Cross Tab Report. This method can be used in SQL Server. In this example, the sales report shown in the first image of this article will be created. The year of the sale will run along the vertical axis, and the quarter for that year along the horizontal axis. The following temporary table (temp tables are distinguished with a # sign) will be used:

```

CREATE TABLE #sales
(
YearSold int,
Quarter char(2),
Amount money
)
INSERT INTO #sales (YearSold, Quarter, Amount) values (2003, 'Q1', 1)
INSERT INTO #sales (YearSold, Quarter, Amount) values (2003, 'Q2', 2)
INSERT INTO #sales (YearSold, Quarter, Amount) values (2003, 'Q3', 3)
INSERT INTO #sales (YearSold, Quarter, Amount) values (2003, 'Q4', 4)
INSERT INTO #sales (YearSold, Quarter, Amount) values (2004, 'Q1', 5)
INSERT INTO #sales (YearSold, Quarter, Amount) values (2004, 'Q2', 6)
INSERT INTO #sales (YearSold, Quarter, Amount) values (2004, 'Q3', 7)
INSERT INTO #sales (YearSold, Quarter, Amount) values (2004, 'Q4', 8)
INSERT INTO #sales (YearSold, Quarter, Amount) values (2005, 'Q1', 9)
INSERT INTO #sales (YearSold, Quarter, Amount) values (2005, 'Q2', 10)
INSERT INTO #sales (YearSold, Quarter, Amount) values (2005, 'Q3', 0)
INSERT INTO #sales (YearSold, Quarter, Amount) values (2005, 'Q4', 0)

```

It is the CASE statements ability to filter that will help sum the sales numbers into the correct quarter. Rather than change a state abbreviation to a name, this time CASE will sum based on the WHEN condition.

```

SELECT YearSold,
SUM(CASE Quarter WHEN 'Q1' THEN Amount ELSE 0 END) AS Q1,
SUM(CASE Quarter WHEN 'Q2' THEN Amount ELSE 0 END) AS Q2,
SUM(CASE Quarter WHEN 'Q3' THEN Amount ELSE 0 END) AS Q3,
SUM(CASE Quarter WHEN 'Q4' THEN Amount ELSE 0 END) AS Q4
FROM #sales
GROUP BY YearSold

```

The statement produces the grid expected, with Year and Quarter as an axis, and Amount summed. The GROUP BY command was included so only quarters for each unique year would be summed for that given year. The keyword ELSE was included so that nulls would not be summed.

YearSold	Q1	Q2	Q3	Q4
2003	1.00	2.00	3.00	4.00
2004	5.00	6.00	7.00	8.00
2005	9.00	10.00	0.00	0.00

PIVOT

The PIVOT keyword is new in SQL 2005. It was designed for pivots and its syntax is more readable than a group of CASE statements. This next statement produces a report identical to the proceeding CASE Cross Tab. In the code below, SUM specifies the grid data and FOR dictates the horizontal axis, IN creates a filter for SUM to match.

```

SELECT *
FROM #sales
PIVOT
(
SUM(Amount)
FOR Quarter
IN (Q1, Q2, Q3, Q4)
)
AS p

```

The letter "p" is used as a table alias for the derived table PIVOT creates. Notice there is no "GROUP BY" clause. The PIVOT command does an automatic group by of all columns not listed in SUM or FOR. In this example, the only column missing is YearSold, which happens to be the column we want grouped. However, if we added a column to our table, StoreLocation for example, the results would be incorrect for our purposes. This next example shows the output from the PIVOT statement above, when two store locations are added:

YearSold	StoreLocation	Q1	Q2	Q3	Q4
2003	IL	NULL	2.00	NULL	4.00
2004	IL	NULL	6.00	NULL	8.00
2005	IL	NULL	10.00	NULL	0.00
2003	WI	1.00	NULL	3.00	NULL
2004	WI	5.00	NULL	7.00	NULL
2005	WI	9.00	NULL	0.00	NULL

PIVOT automatically added StoreLocation to the GROUP BY along with YearSold. To obtain our original results, with the Amount only grouped by YearSold and StoreLocation ignored, the SELECT must create a derived table of only the columns PIVOT should look at. The example below produces the results expected.

```
SELECT *
FROM (SELECT YearSold, Quarter, Amount FROM #sales) as s
PIVOT
(
    SUM(Amount)
    FOR Quarter
    IN (Q1, Q2, Q3, Q4)
)
AS p
```

As an alternative, a Common Table Expression could be used. Common Table Expressions (CTE), are new in SQL 2005 and are used to create temporary result sets. See the Database Journal article "[Common Table Expressions by Don Schlichting](http://databasejournal.com/features/mssql/article.php?3502676)" for more on CTEs. Below is the same pivot but using a CTE to control the automatic group by.

```
WITH MyCTE(YearSold, Quarter, Amount) AS
(
    SELECT YearSold, Quarter, Amount
    FROM #sales
)
SELECT *
FROM MyCTE
PIVOT
(
    SUM(Amount)
    FOR Quarter
    IN (Q1, Q2, Q3, Q4)
)
AS p
```

Conclusion

Pivots can be used to present data in a readable report format. Now with SQL 2005, there are two commands for generating them, PIVOT and CASE.

